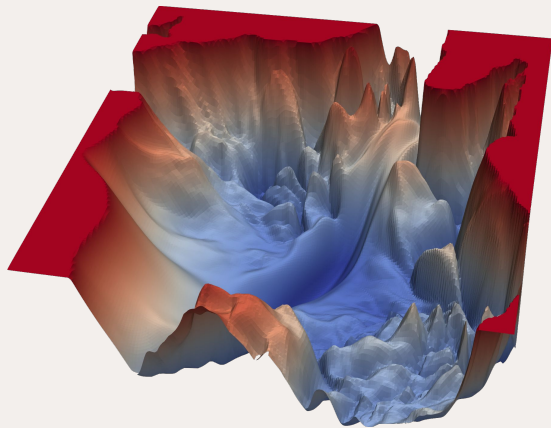


From Memorization to Reasoning in the Spectrum of Loss Curvature



Paper:

Jack Merullo et al.
[@ Goodfire AI](#)

[arXiv:2510.24256](#)

Guest speaker:

Teo Lamort de Gail

Agenda

The “What”

- Memorization, generalization, who cares?
- Existing approaches
- Functional overview

The “How”

- Interpreting loss curvature
- * The Hessian and the surgery
- K-FAC
- Algorithm

More math



Results

Questions

* *At this point, we will understand the title.*

The “What”

Loose definition

Memorization is the tendency of a language model to repeat training sequences word-for-word.

Why do we care?

Copyright

The Times Sues OpenAI and Microsoft Over A.I. Use of Copyrighted Work

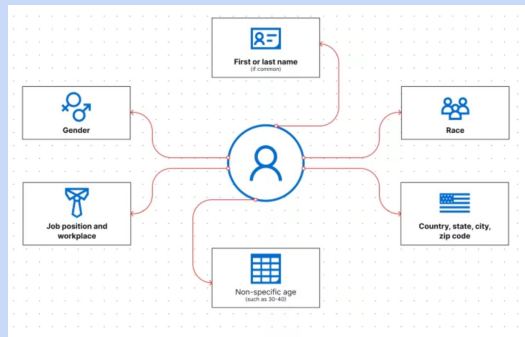
Millions of articles from The New York Times were used to train chatbots that now compete with it, the lawsuit said.

Model size

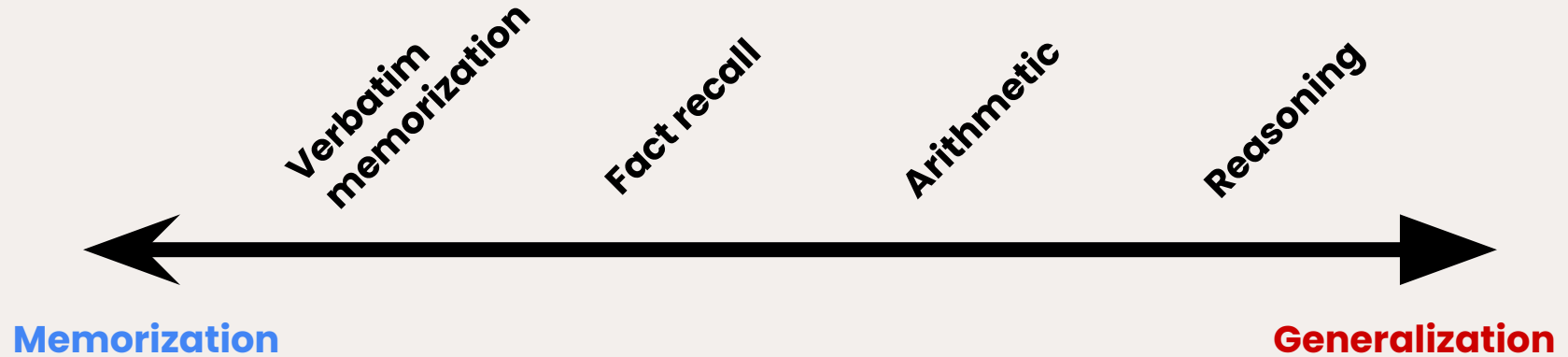
“Current models [...] are forced to waste capacity on memorization.”

– A. Karpathy

Personally Identifiable Information



Philosophy point



Existing approaches

	How it works		
Balanced Subnet (BSN)	Train a mask to suppress specific sequences.	Effective on the targeted data.	Requires a list of what to forget.
Singular Value Decomposition (SVD)	Prune "low importance" weights.	Unsupervised.	Hurts model quality.
Fine-tuning	Train the model to refuse to output copyrighted text.	Practical.	Jailbreaks still possible.

Proposed method



No “unlearning set” needed.

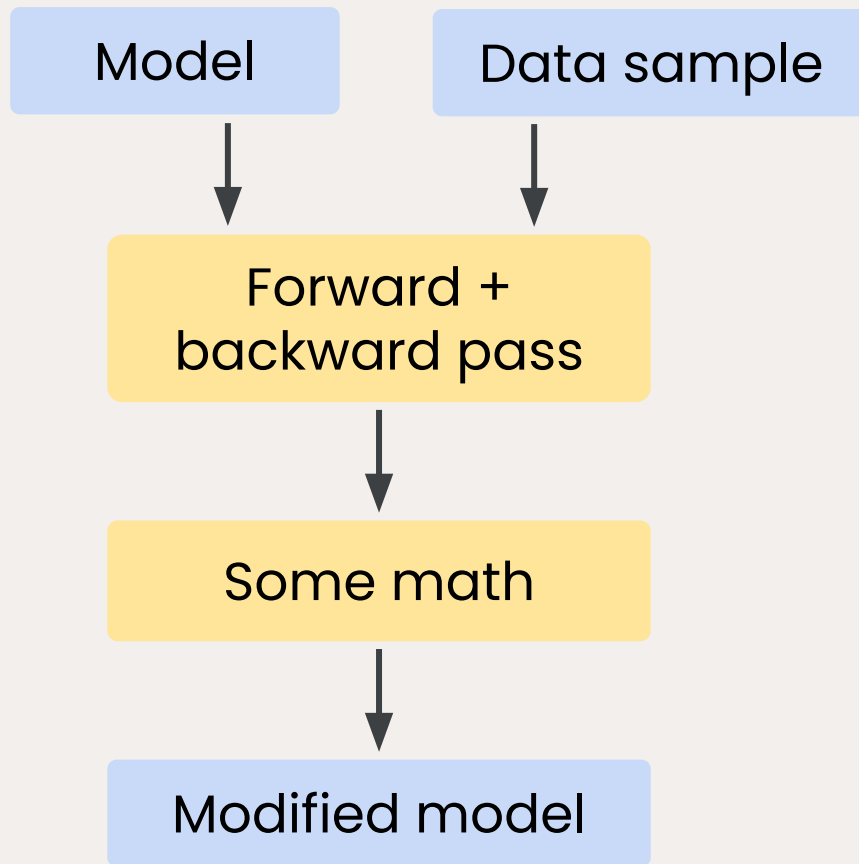


Preserves **reasoning**,
open-book retrieval.



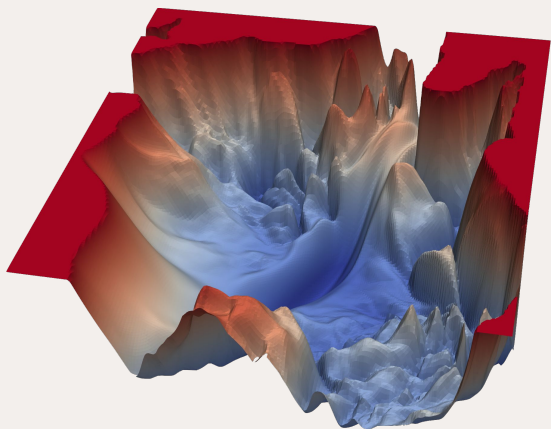
Affects **arithmetic**,
closed-book retrieval.

Functional overview



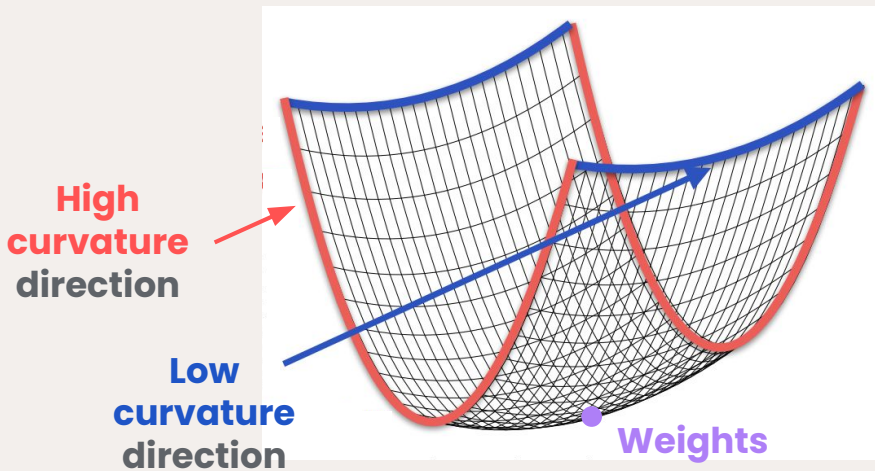
The “How”

Loss landscape



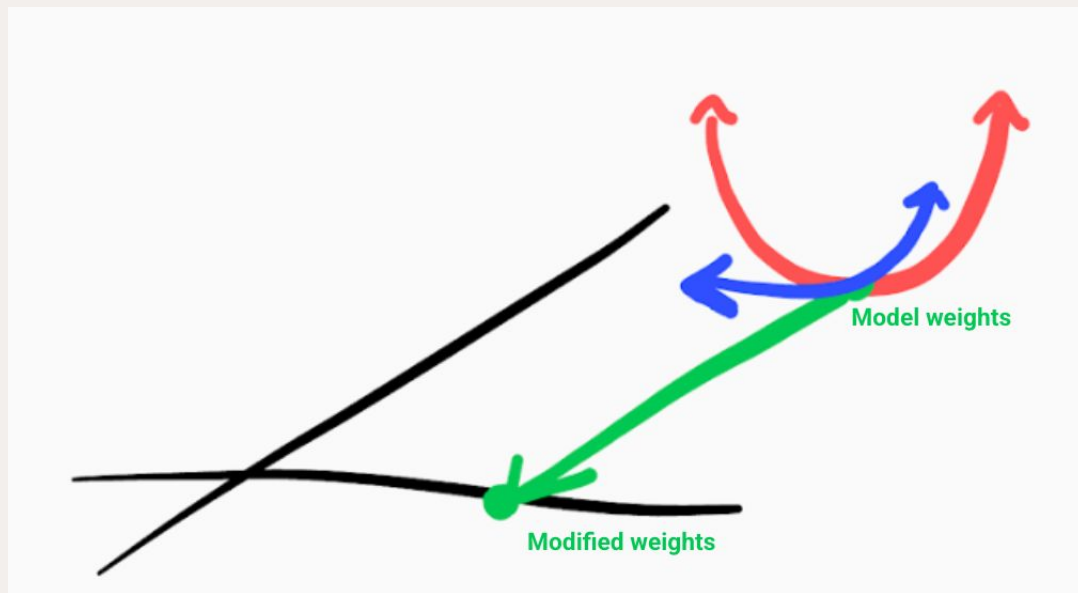
- Loss as a function of model weights.
- Computed **with respect to some data** (entire dataset, single sample...).
- After training, the model is at a **local minimum**.

Interpreting loss curvature



- “If we move along $\mathbf{v}$, how fast does loss increase?”
- Previous work:
At the **sample level**,
high curvature $\leftrightarrow$ memorization.
- Hypothesis:
At the **dataset level**,
low curvature $\leftrightarrow$ memorization.

The surgery: Move in low-curvature directions

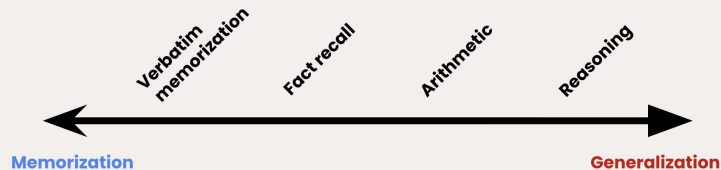


Curvature in higher dimensions

- In the Hessian matrix
Eigenvalues $\leftrightarrow$ **Curvatures**
Eigenvectors $\leftrightarrow$ **Directions**
- Surgery equation:
$$W' = W - (W \cdot v_i)v_i$$
- **We now understand the title of the paper!**

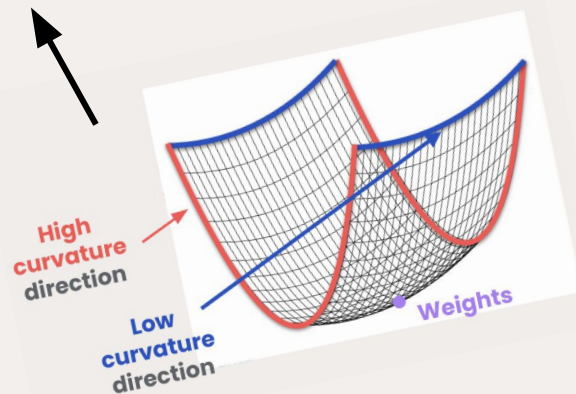
$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}.$$

← flattened model weights →



From Memorization to Reasoning in the Spectrum of Loss Curvature

Eigenvalues $\leftrightarrow$ Curvatures
Eigenvectors $\leftrightarrow$ Directions



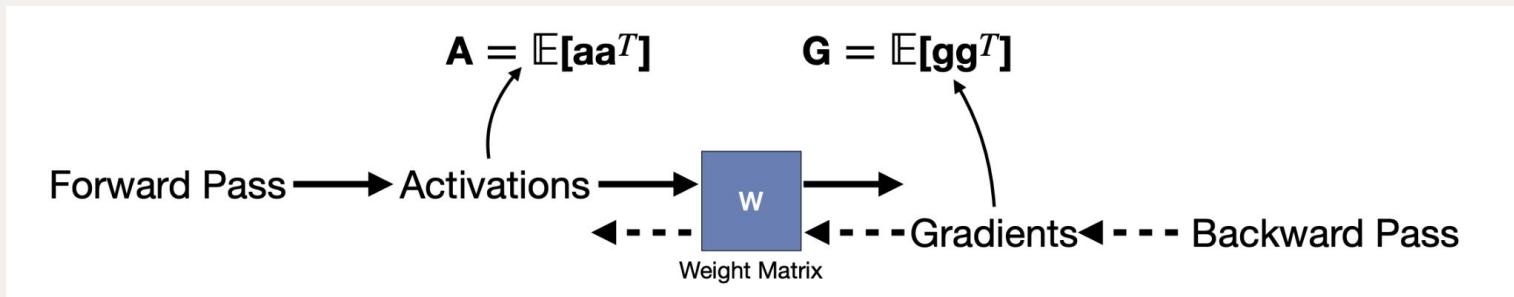
(This is the slide with the most arrows.)

The Hessian is too large

$$(100 \text{ M params})^2 = 40 \text{ PB}$$

K-FAC

or Kronecker-Factored Approximate Curvature



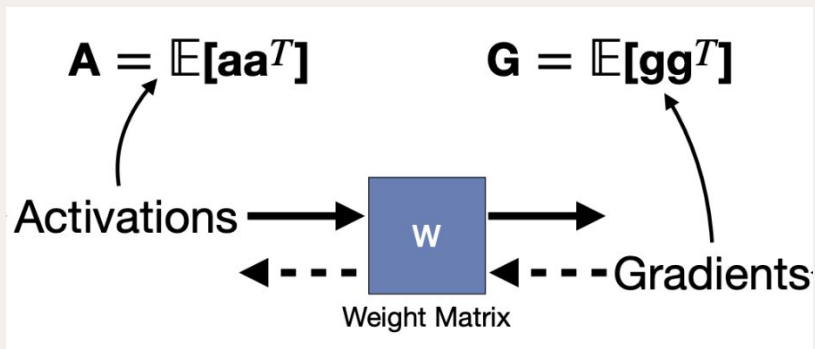
$$H_W \approx G \otimes A$$

Kronecker product:

All pairwise products of elements.

$$[G \otimes A]_{i_1 i_2 j_1 j_2} = G_{i_1 i_2} \times A_{j_1 j_2}$$

Dimension check



$$\begin{aligned} W &: d_{in} d_{out} \\ H_W &: d_{in}^2 d_{out}^2 \\ \mathbf{a} &: d_{in} \\ \mathbb{E}[\mathbf{a}\mathbf{a}^T] &: d_{in}^2 \\ \mathbf{g} &: d_{out} \\ \mathbb{E}[\mathbf{g}\mathbf{g}^T] &: d_{out}^2 \\ \mathbf{G} \otimes \mathbf{A} &: d_{in}^2 d_{out}^2 \end{aligned}$$



Spectrum of $G \otimes A$

If:

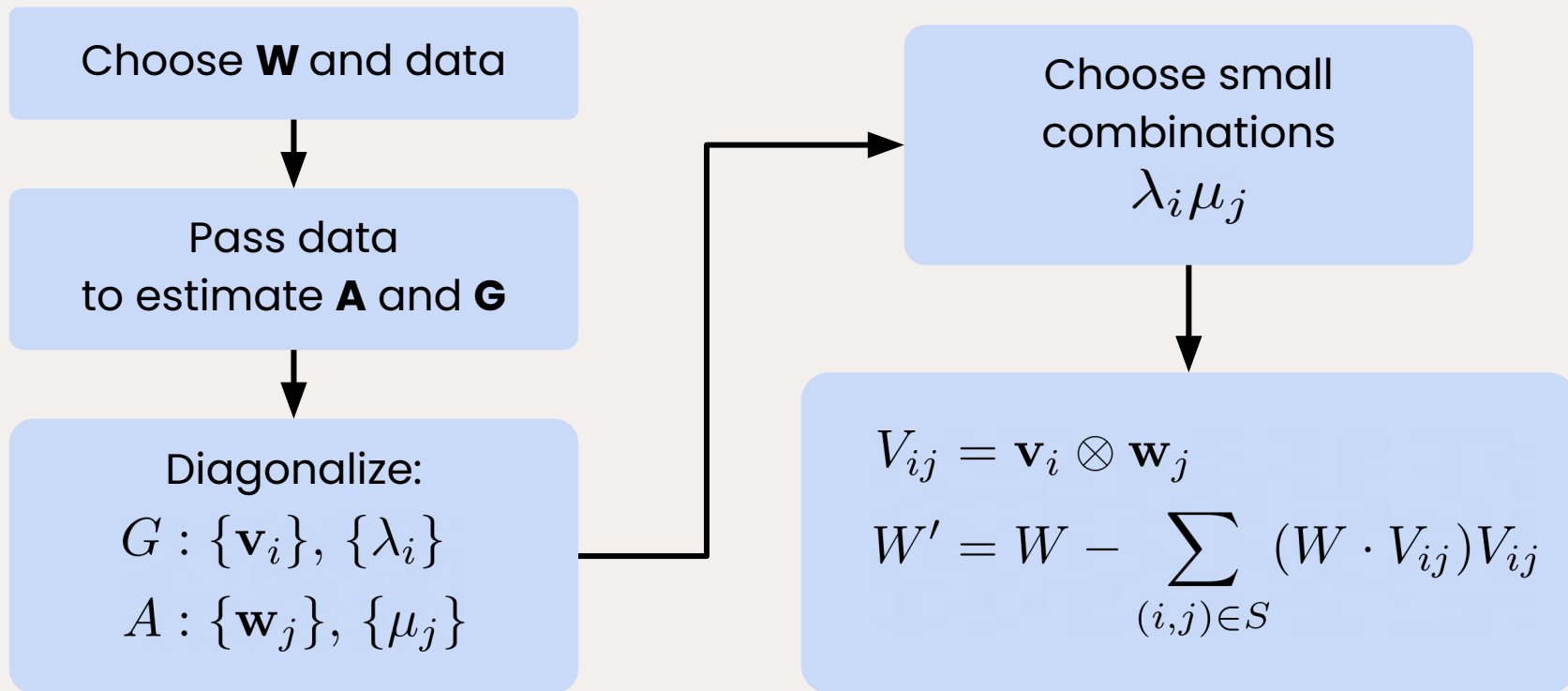
G – vectors: $\{\mathbf{v}_i\}$, values: $\{\lambda_i\}$ ($1 \leq i \leq d_{out}$)

A – vectors: $\{\mathbf{w}_j\}$, values: $\{\mu_j\}$ ($1 \leq j \leq d_{in}$)

Then, very conveniently:

$G \otimes A$ – vectors: $\{\mathbf{v}_i \otimes \mathbf{w}_j\}$, values: $\{\lambda_i \mu_j\}$

TL;DR – Concrete steps



Results

Metrics

Identify memorized data

Prefix/suffix pairs with
perfect greedy decoding,
deduplication



Edit distance post greedy decoding

Model quality guardrail

Perplexity on Pile10k

Experimental setup

Model

OLMo-2 (1B, 7B)

Data

Random subset of training corpus

Hyperparameters

(found via grid search)

Edited layers

MLP 23, 24, 25
(out of 32)

Subtracted components

60% curvature preserved

Results

Explicitly unlearned
(or parameter calibration)

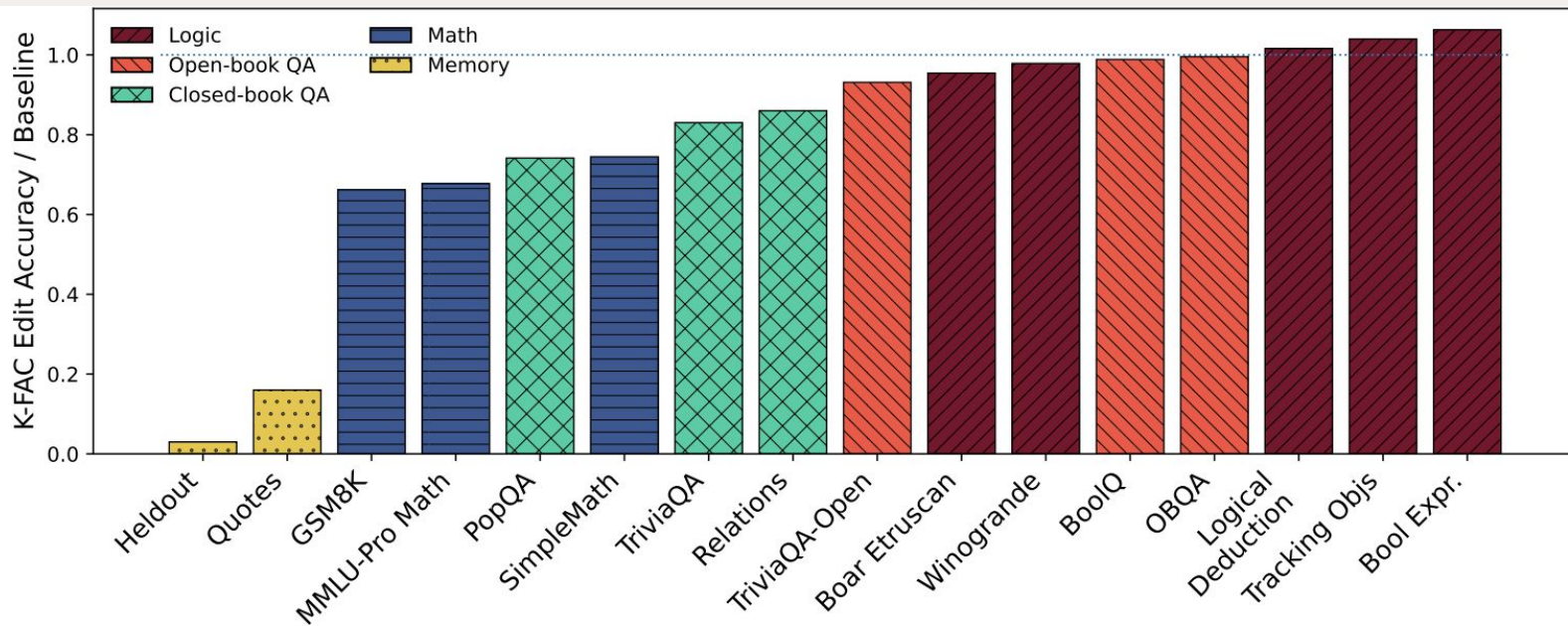
Held out

Model quality

Method	Dolma Validation			Historical Quotes			Pile10k
	Strict (%)	Loose (%)	Avg Lev ↑	Strict (%)	Loose (%)	Avg Lev ↑	Perplexity ↓
7B Model							
Baseline	99.9	100.0	0.002	99.9	100.0	0.001	19.04
BSN	6.0	11.0	0.860	60.0	79.0	0.180	23.59
K-FAC	3.4	8.8	0.704	16.1	23.8	0.625	22.84
SVD	3.0	6.8	0.754	17.5	30.4	0.560	22.49

Unsupervised
memorization suppression

Downstream performance

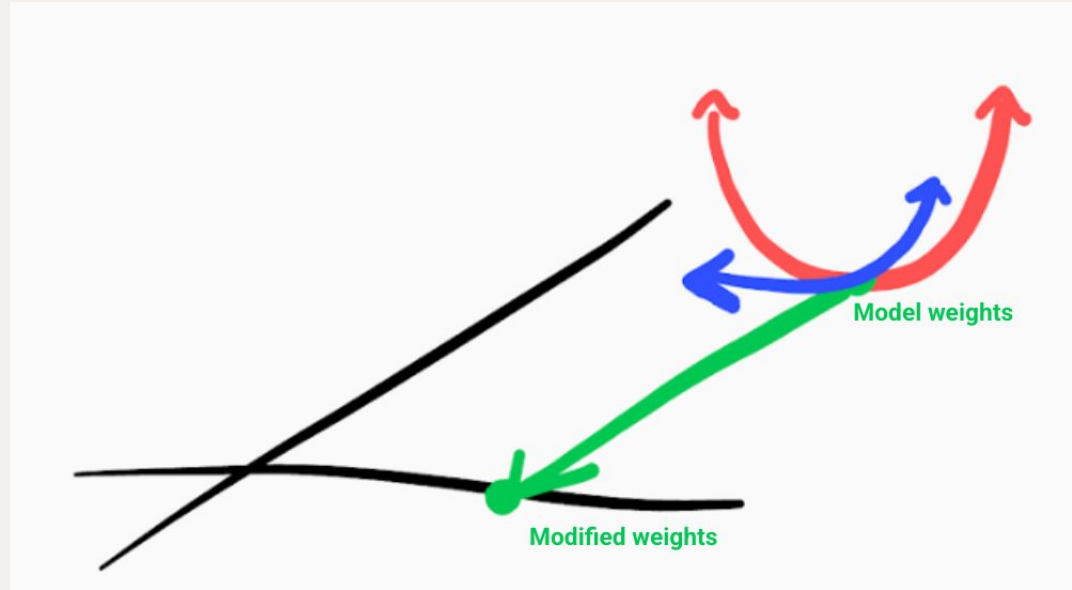


Memorization

Generalization

Questions?

Opinion: Should explore smaller perturbations

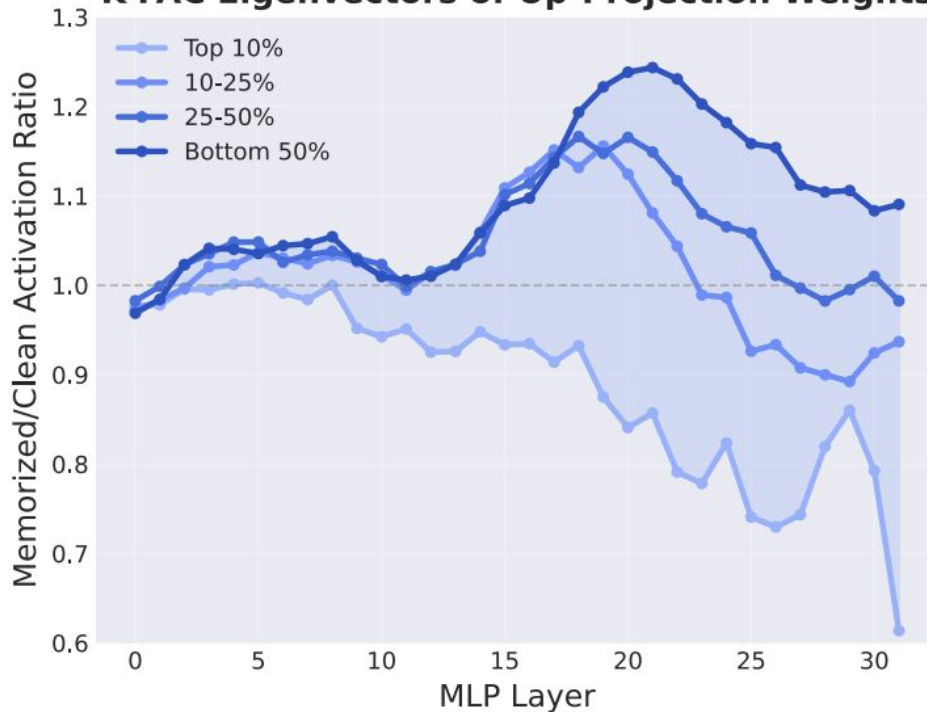


Bonus slides

Additional experiment: Activation ratio

$$\frac{\|Cx_{mem}\|}{\|Cx_{clean}\|}$$

Activation Ratio between Memorized and Clean Text:
K-FAC Eigenvectors of Up-Projection Weights



Memorized data selection

	Dolma	Historical Quotes
Prefix Suffix	64 48	L - 8 8
Extra filtering?	✓	✗
Num. for training or grid search	500	-
Num. for eval	500	512

K-FAC vs SVD

- Better generalization to unseen data.
- K-FAC retains full matrix rank:
more “theoretically grounded”.
- SVD is known for inflated perplexity.
- K-FAC flat-out better in ViT experiment.

K-FAC exactness

- K-FAC actually approximates the Fisher Information Matrix, using the independence assumption:

$$\mathbb{E} [A \otimes G] \approx \mathbb{E} [A] \otimes \mathbb{E} [G]$$

- The FIM is exactly the expected Hessian of the “self-loss”:

$$\mathcal{L}_{\text{self}}(x) = \mathbb{E}_{y \sim p_{\theta}(y|x)} [-\log p_{\theta}(y|x)]$$

- The “self-loss” is $\approx$ the true loss for a good model.

Thank you!